

Rover Houston — Technical Writeup

Team PoliSa

2026-05-09

Houston: An On-Device Multi-Agent AI Habitat Controller for Mars

Team PoliSa · GDG AI Hack 2026 · MSI “Cut the Cord” track Hardware: MacBook Pro · Apple M3 Pro · 18 GB unified memory (Min tier) Repository: <https://github.com/landigf/gdgaihack-2026> · branch feat/houston-rag

1. The brief, taken literally

Cut the Cord rewards projects “where local AI is the **correct architecture**, not just a constraint.” We chose the strongest possible structural answer: **Mars**. Earth-Mars one-way light time is 4–24 minutes; a 14-day comms blackout fires every 26 months at solar conjunction; the closest data center is ~78 million km away. A cloud SLA cannot reach the operator. We didn’t pick offline — physics did.

Houston is the AI assistant we built for that operator. It runs on a single laptop. Hold airplane mode on, drop the network, and every demo loop continues to work — verified by `scripts/airplane-check.sh` (exit 0) and a `tcpdump -i any -nn` window during the pitch (0 outbound packets in 90 s).

2. Architecture overview

— Tauri 2 desktop shell (Rust + WebKit webview) —

```
React 18 + Vite 6 + Tailwind 3 (renderer) — ~340 MB
├─ /rover   : Rover Core file Finder (Stream A's MVP)
├─ /ares    : Rover Houston · Mars Habitat (Stream B — this writeup)
│   └─ Isometric Mars base (react-three-fiber, 7 GLB buildings)
│   └─ Greenhouse drill-in: 4-shelf wire-mesh rack, 16 individual pots,
│       per-species procedural plants × 6 growth stages
│   └─ Inventory drill-in: 6 life-support bars + 4-crew roster + survival
│   └─ Floating Push-To-Talk (browser MediaRecorder)
│   └─ Live perf footer (FPS, CPU%, RAM, sidecar/Ollama RSS)
Python FastAPI sidecar (uvicorn @ 127.0.0.1:8765) — ~95 MB
├─ Rover Core: /index, /search, /summarize (FAISS + nomic-embed-text)
└─ Houston   : /ares/houston/greenhouse · /survival · /voice/houston · /perf

whisper.cpp (`/opt/homebrew/bin/whisper-cli`) — ggml-base.en.bin · 141 MB
macOS native `say` (built-in, zero install) — TTS Daniel voice
Ollama @ localhost:11434 — gemma3:4b Q4_K_M (3.5 GB) + nomic-embed-text (274 MB)

Mars corpus (one-time setup): 30 NASA public-domain PDFs · 154 MB on disk
├─ NASA-STD-3001 Vol 1 (Crew Health), HRP Evidence Book, ISS Medical Checklist
└─ Veggie + Advanced Plant Habitat (APH PH-04) fact sheets
```

└ EVA procedures, Mars DRA 5.0
Indexed locally: 30 files → 1 292 chunks → FAISS · 3.9 MB index

▲ all inference local · zero remote requests · airplane-check exit 0

3. Multi-agent design (A2A done honestly)

Houston is a **two-agent agentic system** where both agents share the same Ollama process and the **bytes-identical** system-prompt prefix HOUSTON_PREFIX so the second call's prompt-eval reuses the cached prefix tokens. This is the brief's "smart caching" beat measured, not claimed.

Persona	Purpose	Trigger
greenhouse	Plant decisions per pot — strict verdict priority (stage 5 □ HARVEST NOW; in-range sensors □ NOMINAL; out-of-range □ CORRECT). 1–2 sentence imperative narration with [S_n] citations.	User clicks any pot in the greenhouse drill-in.
procedure	Converts the verdict into a 3–5 step imperative checklist; cites the same chunks.	Auto-chained right after greenhouse returns.
survival	Habitat / ECLSS inventory & life-support tip with severity (ok/watch/critical).	User clicks Habitat or ECLSS on the base map.
voice	Capcom-style spoken reply (max 2 sentences, no [S_n] in spoken text).	User holds the PTT button.

All four personas reuse HOUSTON_PREFIX. The model is loaded once (keep_alive=30m) and never reloads during the demo. We measured the chained Procedure call versus a fresh request to gemma3:4b: see a2a_kv_cache.png.

Hybrid retrieval ladder

Houston doesn't always ask the LLM. Each call follows the cheapest tier that answers:

1. **Tier 0 — direct simulator state** (sensor readings, inventory): pure Python, ~0.1 ms.
2. **Tier 1 — RAG retrieval** (FAISS + nomic-embed-text, top-3): ~30–60 ms, no LLM tokens.
3. **Tier 2 — LLM with grounded context**: [S_n] chunks injected into the user message; verdict is JSON-parsed and citation IDs are validated against the retrieved pool (we reject invented [S_n]).
4. **Tier 3 — A2A chain** (Procedure persona) only when the user is on the greenhouse drill-in.

If Ollama or the corpus is unavailable, every endpoint falls back to a deterministic stub with placeholder citations. The frontend shows ● LIVE LLM vs ○ FALLBACK so the demo never silently lies.

4. Optimization for M3 Pro 18 GB

The Min tier is 16 GB and we are at 18 GB — there is no headroom for a 7B+ model. So we explicitly engineered around the hardware:

- **Model choice**: gemma3:4b Q4_K_M (~3.5 GB RSS) over qwen3:4b (which has thinking-mode preamble that doubles latency without quality gain on this corpus). Override path retained via GEN_MODEL env so M4 Pro 24 GB can use the larger model.
- **Embedding co-tenancy**: nomic-embed-text (137M, F16) lives in the same Ollama process. Single port, single GPU context, no second backend to evict.

Houston endpoint latency cold vs warm

Figure 1: Houston endpoint latency cold vs warm

Voice round-trip breakdown

Figure 2: Voice round-trip breakdown

- **Whisper.cpp + Metal**: ggml-base.en (~290 MB) loaded lazily only on first PTT press; subsequent calls keep the model warm in the OS file cache. **Cold ASR 2.6 s, warm 0.6 s** on a 4-second utterance.
- **macOS say for TTS** instead of Piper or Kokoro: ships in the OS, zero install, no extra RAM for an inference engine, no CDN dependency. Sounds appropriately “mission-control” for the demo.
- **Procedural 3D over GLBs when bundled assets fail**: `<Suspense fallback=<ProceduralBase />>` means the demo never shows an empty pad if a GLB fetch fails — graceful degradation under the same offline contract.
- **Renderer dpr cap = [1, 2]** + `powerPreference: "high-performance"` so R3F never burns the iGPU on a 4K display.
- **psutil-sampled /ares/perf** at 2 Hz drives the live FPS / CPU / RAM footer — judges can verify on stage.

5. Measured numbers (reproducible)

All numbers below come from `python benchmarks/houston/run_benchmarks.py` running against the live sidecar on **the demo machine** with the Mars corpus indexed (1 292 chunks). CSVs + plots saved under `benchmarks/houston/out/`.

Workload	Cold	Warm median	P95 (warm)
/ares/houston/greenhouse (greenhouse + procedure A2A)	8 745 ms	8 124 ms	8 192 ms
/ares/houston/survival (single persona, RAG)	3 341 ms	3 599 ms	—
/ares/voice/houston (whisper □ LLM □ say, full round-trip)	3 117 ms	1 820 ms	—

Resource (idle □ load)	Idle	Houston greenhouse + A2A
RAM used	8.14 GB / 18 GB (44 %)	10.17 GB / 18 GB (55 %)
Sidecar RSS (Python)	~95 MB	~110 MB
Ollama RSS	~110 MB pre-warm	~3 700 MB warm
FPS (R3F isometric scene, idle)	≥ 55	≥ 45 during LLM call

Plots (committed under `benchmarks/houston/out/`)

Figure 1 — Cold vs warm latency across the three Houston endpoints. The cold call carries the gemma3:4b model load; subsequent warm calls amortize over the `keep_alive=30m` window.

Figure 2 — Warm voice round-trip stacked: whisper.cpp ASR + gemma3:4b LLM + macOS say TTS. Total under two seconds end-to-end on M3 Pro 18 GB.

Multi-agent A2A KV-cache reuse

Figure 3: Multi-agent A2A KV-cache reuse

Perf timeline during a Houston call burst

Figure 4: Perf timeline during a Houston call burst

Figure 3 — Two-persona A2A chain (greenhouse □ procedure). The second call is consistently faster because the system prompt prefix is bytes-identical and the KV cache is reused.

Figure 4 — CPU % and RAM (GB / 18 ceiling) sampled at 1 Hz during a five-call greenhouse burst. The 18 GB hard ceiling is shown as a dashed line; load peak stays at ~10.2 GB.

6. What we'd do next (engineering, not pitch)

1. **MLX backend** for whisper instead of whisper.cpp. M3 Pro's ANE is currently idle during ASR; MLX-Whisper benchmarks show ~3× speedup.
2. **Move the Procedure persona to the chat endpoint** with explicit `keep_alive` + `messages[]` history so we get true KV-cache reuse signals from Ollama (currently inferred from the prompt-eval-time delta heuristic).
3. **Streaming token output** for Houston narration — first token in <300 ms instead of waiting 1.8 s for a 200-token reply.
4. **Background indexer** — `current / index` is sync and blocks the renderer for 3 minutes on the 30-PDF corpus. Move to a worker thread with progress events on the existing FastAPI WebSocket scaffold.
5. **Real Mars latency simulator** that gates outbound voice replies behind the synodic-modeled round-trip; would turn the demo from “look how fast” into “look how the operator handles delay.”
6. **Multi-modal vision input** — `gemma3:4b` is multimodal; we could wire a MacBook FaceTime camera frame into the greenhouse persona and have it cross-validate the NDVI sensors against an actual image.

7. What does NOT work yet (honesty section)

- Sensor data is **simulated**, not from real spectral hardware. The simulator follows NASA Veggie/APH envelopes and the demo banner declares “DEMO MODE · plant cycles accelerated 100×”.
- Drill-ins exist for **Greenhouse, Habitat, ECLSS**. The other 4 buildings (ISRU, Power, Airlock, Rover Garage) remain hover-tooltip targets — explicit in the iteration plan.
- The Mars latency chip uses a **simplified synodic approximation**, not real ephemeris.
- `cpu_temp_c` is not reported on Apple Silicon (`osx-cpu-temp` is Intel-SMC-only); reading via `powermetrics` requires `sudo` and is left to a future setup hook.

8. Compliance with MSI Cut the Cord (the binary check)

Rule	How we satisfy it
Zero cloud AI at demo time	Ollama (<code>gemma3:4b</code> + <code>nomic-embed-text</code>), <code>whisper.cpp</code> , macOS say — all local.
100 % local inference	<code>scripts/airplane-check.sh</code> greps source for external URLs and exits 0.
Software integration with the OS	Verified via <code>tcpdump -i any -nn</code> during a 90-second demo: 0 outbound packets.
Single machine	Tauri shell opens NASA PDFs in macOS Preview at the cited chunk; native folder picker; mic permission via Tauri capability; macOS say system call.
Airplane-mode test	All 4 agents, ASR, TTS, and the FAISS index run on the same M3 Pro.
	Pass. Demo loop survives Wi-Fi off identically.

Reproduction:

```
git clone https://github.com/landigf/gdgaihack-2026.git
git checkout feat/houston-rag
bash scripts/setup.sh # rust + ollama models + python venv + npm
bash scripts/download-mars-corpus.sh # 30 NASA PDFs, ~5 min
GEN_MODEL=gemma3:4b npm run tauri dev # Houston window opens
python benchmarks/houston/run_benchmarks.py # generate the plots in this writeup
```